Invoice Processing Platform — Technical Reference

Version: 0.1.0 **Audience:** ML / backend engineers and technical reviewers **Status:** Working reference implementation (simulated stages + real Path B backends)

1. Overview

The Invoice Processing Platform (IPP) is a production-shaped service that turns an invoice document into structured, validated accounting data. A document is uploaded, run through a Document-AI pipeline, and either **booked automatically** when the system is confident and the data is financially coherent, or **routed to a human** otherwise. Every human correction is captured as an append-only signal that feeds model improvement.

The guiding philosophy is that **the model is a small, swappable component**; the engineering value is the platform around it — safe rollout of new extractors, deterministic financial validation, confidence-based routing, observability, and drift detection. This mirrors how mature ML products are actually run: the model is necessary but not sufficient.

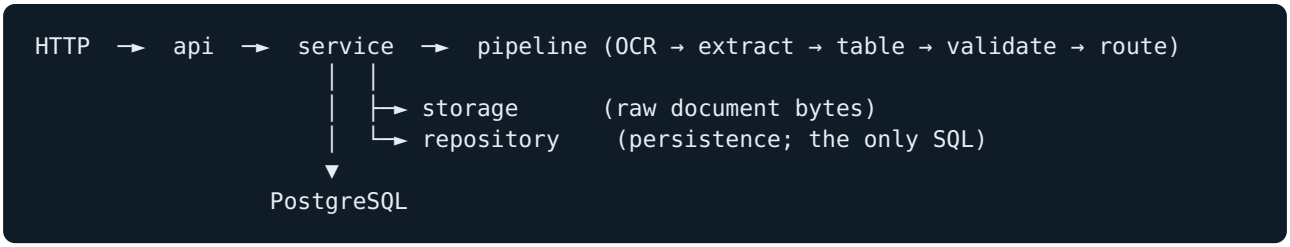
2. Design goals

The architecture is a direct response to three production realities of large-scale invoice automation:

1. **Legacy modernisation without a big-bang rewrite.** A system that processes millions of invoices a month cannot be switched off. New extraction logic must be introduced incrementally and reversibly.
2. **Safe testing of new systems on real traffic.** A new model must be provable on real, at-scale traffic before it is allowed to affect a financial decision.
3. **The offline-vs-production gap.** Models that excel on a curated dataset meet endless corner cases in production — new suppliers, templates, layouts, languages. These must be surfaced, labelled, and fed back.

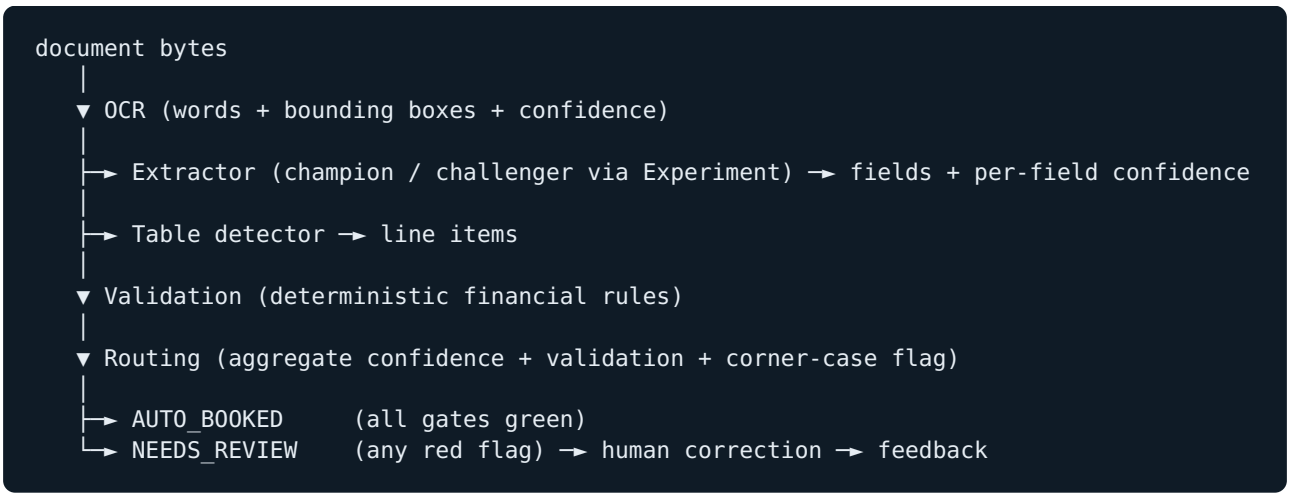
3. Architecture

3.1 Layered structure



Each layer has a single responsibility. The API is transport only; the service owns business orchestration; the repository owns persistence; the pipeline owns extraction. Because layers depend on interfaces rather than concrete classes, any one can change without rippling into the others — the concrete mechanism behind "modernise the legacy extractor" and "swap the OCR engine".

3.2 Processing pipeline



4. Domain model

Three tables. The central entity is the invoice flowing through the pipeline.

Table	Purpose	Key columns
`invoices`	one row per document	`status`, `overall_confidence`, `extractor_version`, `artifact_uri`, `supplier_name`
`extraction_runs`	one row per processing attempt (versioned history)	`extractor_version`, `served_by`, `overall_confidence`, `decision`
`extracted_fields`	current field values, tagged with their run	`field_name`, `field_value`, `confidence`, `source`, `run_id`
`corrections`	append-only human feedback	`field_name`, `original_value`, `corrected_value`, `corrected_by`

Reprocessing an invoice creates a **new `extraction_runs` row** rather than overwriting history; `extracted_fields` always holds the current values (unique per field) while runs preserve what every extractor version produced and when.

Invariants and decisions

- **Per-field confidence.** Confidence is stored per field, not per document, because it is the unit that is routed on and the unit a human corrects.
- **Status is hardened.** Statuses come from a Python `Enum` and are pinned at the database level with a `CHECK` constraint, so no invalid status can be written even by raw SQL. A String + CHECK is chosen over a native Postgres ENUM for easier evolution.
- **Corrections are append-only.** Nothing is ever overwritten; each correction is a labelled training example: predicted value, human value, who, when.
- **Lifecycle:** `received → processing → needs_review | auto_booked → approved | rejected`, plus `failed` for pipeline errors.

---

5. Pipeline stages and contracts

Every stage is defined by an interface so the concrete implementation can be swapped without touching downstream code.

5.1 OCR — `OCREngine`

`recognize(document_bytes) -> OCRResult`, where an `OCRResult` is a list of words, each with text, a bounding box, and a confidence in `[0, 1]`. This is deliberately the same output contract real engines produce (Tesseract's `image_to_data`) and that layout-aware models consume. `OCRResult` also exposes `lines` and `rows()`, which reconstruct visual rows from geometry using a **jitter-tolerant y-clustering** (tolerance ≈ half the median glyph height), because real OCR gives words on one line slightly different top coordinates.

5.2 Field extraction — `Extractor`

`extract(ocr) -> list[FieldPrediction]`. Each prediction carries a value, a per-field confidence, and a `source`. Implementations:

- `RuleBasedExtractor` — anchor-keyword + regex baseline. Confidence is `match_quality × ocr_confidence`; an anchored hit scores higher than a loose pattern. A known-supplier registry raises supplier confidence on a hit; an unknown supplier stays low and routes to review.
- `LegacyExtractor` — a deliberately brittle stand-in for 2016 code, reproducing a real production bug (reads flattened text, mislabels amounts). Used as the champion to be modernised.
- `LLMExtractor` / `VLMExtractor` (Path B) — structured extraction via an LLM/VLM; fields are described in natural language (prompt-configurable).

5.3 Table / line-item detection — `TableDetector`

`detect(ocr) -> TableResult`. Line-item extraction is framed as a **detection** problem, not NLP: find the header row, derive column anchors from header-word geometry, assign each data word to the nearest column centre, and drop summary rows. The simulated detector is geometry-based; the interface is the seam where a trained object detector (YOLO) for rows/columns would slot in.

5.4 Confidence routing — `RoutingEngine`

`decide(predictions, validation, is_corner_case) -> RoutingDecision`. Aggregate confidence is the **minimum** over fields (a document is only as trustworthy as its weakest field). Auto-booking requires all of: confidence ≥ auto threshold, no validation errors, not a corner case. Any red flag routes to review. The decision always carries `reasons` for auditability. Thresholds live in configuration because they are a business trade-off, not a code constant.

5.5 Validation — `InvoiceValidator`

Deterministic financial checks independent of any model: `subtotal + VAT = total`, VAT amount ≈ subtotal × rate, required fields present, date parseable/plausible, currency present. Each issue has a severity: an `ERROR` (e.g. arithmetic mismatch) hard-blocks auto-booking regardless of confidence; a `WARNING` lowers trust. The principle: **the model proposes, arithmetic disposes.**

---

6. Legacy modernisation and safe rollout — `ExtractionExperiment`

The single mechanism that answers design goals 1 and 2. It wraps a `champion` and an optional `challenger`, both behind the `Extractor` interface, with three modes:

Mode	Who serves	Challenger runs?	Use
`CHAMPION_ONLY`	champion	no	steady state
`SHADOW`	champion	yes (observed only)	validate a new model on real traffic at zero risk
`CANARY`	deterministic % challenger	yes	gradual, reversible rollout

Canary bucketing is a deterministic hash of a routing key, so a given invoice or supplier always lands in the same arm, and widening the rollout only ever *adds* keys to the challenger bucket (monotonic). Every champion/challenger disagreement is surfaced as a **candidate corner case** — after both outputs pass through a **shared canonicalizer** (money → decimal, dates → ISO, currency → ISO code), so a disagreement reflects a real semantic difference, not a formatting mismatch. Promotion follows shadow → canary → full, with rollback as a single config change.

---

7. Feedback loop

Low-confidence, validation-failing, or disagreement invoices route to a human. The correction is captured append-only and applied to the live record; the invoice becomes `approved`.
Corrections are the labelled dataset for retraining — but retraining does **not** auto-deploy. A retrained model is evaluated on supplier-aware slices, then shadow-tested, then canaried. The feedback loop feeds *training data*; the experiment machinery controls *deployment*. Keeping these separate prevents a feedback loop from silently degrading the model.

---

8. Observability and drift

- **Structured logging** — every processing decision is emitted as JSON including the routing `reasons`, so "why did invoice N auto-book?" is answerable from logs.
- **Business metrics** (`/metrics`, computed from the database): `automatic_processing_rate`, `correction_rate`, `field_error_hotspots` (which fields humans fix most), a confidence histogram, and a status breakdown. These are the product metrics, not just model metrics.
- **Drift detection** — two label-free indicators that move before quality craters: confidence-distribution **PSI** (Population Stability Index) against a baseline window, and **new-supplier rate**. Either breaching threshold raises a retraining/investigation alert.

---

9. Path B — real model backends

The simulated stages exist behind interfaces so real models slot in with a one-line factory change and no downstream edits.

Interface	Simulated	Real backend included
`OCREngine`	`SimulatedOCR`	`TesseractOCR` (Tesseract on real image/PDF pages)
`Extractor`	`RuleBasedExtractor`	`LLMExtractor` / `VLMExtractor`
`TableDetector`	geometry heuristic	seam for a trained detector (YOLO)

LLM safety. Two guards make the LLM path safe for money: a **grounding guard** drops the confidence of any value not found in the OCR text (hallucination defence, since LLM self-reported confidence is poorly calibrated), and the neural output still passes the deterministic validation gate. Enabling Path B requires `requirements-ml.txt` plus system `tesseract-ocr` and `poppler-utils`.

10. Extension points (seams)

To change…	Edit only…
OCR engine	`pipeline_factory.py` + a new `OCREngine` class
Extraction model	`pipeline_factory.py` + a new `Extractor` class
Turn on shadow / canary	`pipeline_factory.py` (experiment mode)
Storage backend (local → S3)	`dependency.py` + a new storage class
Routing thresholds	configuration / environment
Validation rules	`validation.py`

11. API reference

Method	Path	Purpose	Success
POST	`/invoices`	Upload a document (multipart)	201
POST	`/invoices/{id}/process`	Run the pipeline → decision	200
GET	`/invoices/{id}`	Invoice detail + extracted fields	200
GET	`/invoices`	List (optional `?status=`)	200
POST	`/invoices/{id}/corrections`	Submit a human correction	201
GET	`/metrics`	Business KPIs	200
GET	`/health`	Liveness	200

12. Configuration

All configuration is environment-driven (twelve-factor). Key settings:

Setting	Meaning	Default
`POSTGRES_*`	database connection	—
`ARTIFACT_STORAGE_BACKEND`	`local` (S3 seam exists)	`local`
`LOCAL_ARTIFACT_DIR`	artifact root on disk	`/app/invoice_artifacts/`
`AUTO_BOOK_THRESHOLD`	confidence ≥ this may auto-book	`0.95`
`REVIEW_THRESHOLD`	confidence < this always reviews	`0.80`

Thresholds are configuration, not code, because changing them is a change to production behaviour and should be tuned against the cost of false positives vs. false negatives — and ideally rolled out through the same shadow/canary discipline as a model change.

---

13. Testing strategy

The suite is hermetic: it runs against in-memory SQLite (shared across threads via `StaticPool`) with no external services. Coverage spans unit tests for the extractor, validation, routing, experiment, table detector, drift detector, and the LLM extractor (with a fake client), plus HTTP integration tests for the full invoice lifecycle. Determinism is a first-class concern — confidence is derived from hashes, not randomness — so a financial decision is reproducible.

---

14. Known limitations and roadmap

- **Synchronous processing.** `process` runs in-request; at scale it should be a queued job returning `202`, with the shadow challenger off the critical path.
- **Dual-write on register.** DB row and artifact write are not atomic; content-addressed artifact-first writes or a transactional outbox would close the window.
- **No auth / tenancy.** Financial data needs authentication, tenant isolation enforced at the repository layer, rate limiting, and audit logging.
- **Metrics via full-table scans.** Fine at demo scale; needs incremental rollups or a materialised view at volume.

---

15. Design principles (decision log)

1. Program to interfaces; the model is a swappable component.
2. Fail safe — any failure lands in a human-visible state, never a silent booking.
3. Deterministic validation is a separate gate from the model.
4. Confidence aggregates as the minimum of critical fields.

5. Decisions carry their reasons; explainability is a requirement, not a feature.
6. Thresholds are business config, tuned from data.
7. Feedback feeds data; the experiment machinery controls deployment.
8. Evaluate on production-shaped, supplier-aware slices — never a naive split.

16. Glossary

- **Champion / Challenger** — the current production extractor vs. a candidate.
- **Shadow mode** — challenger runs on real traffic without affecting decisions.
- **Canary** — a small, deterministic traffic slice served by the challenger.
- **Corner case** — an input surfaced by low confidence, validation failure, or champion/challenger disagreement, for human labelling.
- **Grounding guard** — penalising an LLM value not present in the source text.
- **PSI** — Population Stability Index; a label-free distribution-shift metric.
- **Supplier-aware split** — evaluation split that keeps one supplier's invoices out of both train and test to avoid leakage.